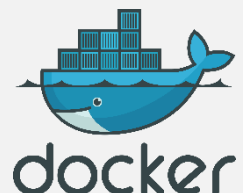


End-to-End MLOps With Google Cloud Platform

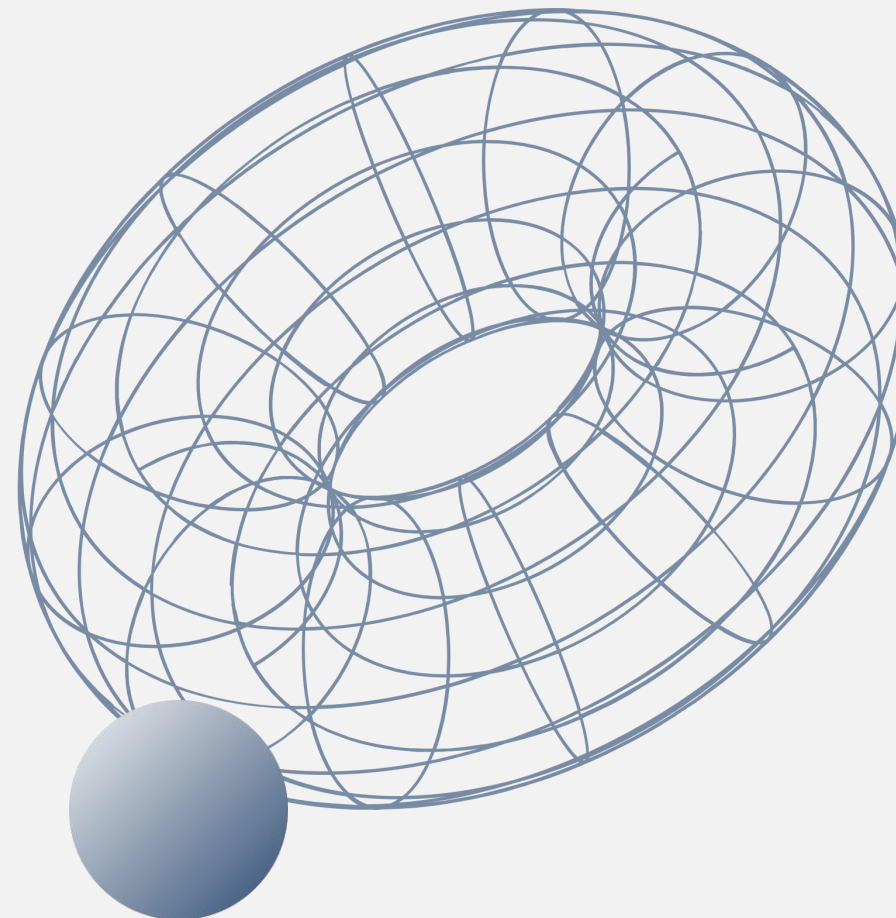
Tech Stack:



Project Structure Details

FLIGHT_PRICE_PREDICTOR_APP/

- app/
 - __pycache__/
 - exception_logging/
 - utils/
 - __pycache__/
 - __init__.py
 - data_cleaner.py
 - data_loader.py
 - feature_engineering.py
 - model_trainer.py
 - __init__.py
 - config.py
 - main.py
 - python_SDK_training_deployment.ipynb
- data/
- logs/
- models/
- notebooks/
- preprocessor/
- processed_batch_data/
- static/
- templates/
- .dockerignore
- .gitignore
- cloudbuild_hpt_train.yaml
- cloudbuild_hpt.yaml
- cloudbuild_serve.yaml
- cloudbuild.yaml
- Dockerfile.serve
- Dockerfile.train
- python_SDK_batch_inference.ipynb
- README.md
- requirements.txt
- run.py
- vertex_hpt.yaml



This project utilizes **object-oriented programming** to create a scalable solution for predicting flight prices. Google Cloud Platform Vertex AI has been used for Model Training, Hyperparameter Tuning and Deployment of best model. Batch Inference is designed using Python gcloud SDK and output is saved in Bucket. Finally, the training pipeline (cloudbuild_hpt_train.yaml) and serving pipeline (cloudbuild_serve.yaml) deployed separately and CI/CD pipeline established via cloud build trigger in synchronization with GitHub Repository.

Workflow

Training Pipeline:

- 1** Build Trigger
Cloud build process triggered via gcloud build command using cloudbuild_hpt_train.yaml file.
- 2** Containerization and push to Artifact Registry
Docker containerize the image (using Dockerfile.train) and push it to Artifact Registry as the first two step of the cloudbuild_hpt_train.yaml file.
- 3** Model Training and Hyperparameter Tuning
Submits Model Training job along with Hyperparameter tuning in Vertex AI.
- 4** Detect the best model and upload to Model Registry
Once the training job is completed, all trail models are saved to bucket. Then Vertex AI detect the best model and upload it to Model Registry.
- 5** Create Endpoint and Deploy the uploaded model to Endpoint
In this step an Endpoint is created and the uploaded model gets deployed to the Endpoint.
- 6** Save the Endpoint Id to Secret Manager
Once the Model is successfully deployed to the Endpoint, the Endpoint Id is saved to Secret Manager.

Serving Pipeline:

- 1** Build Trigger
Build process triggered via gcloud build command using cloudbuild_serve.yaml file.
- 2** Containerization and push to Artifact Registry
Docker containerize the image (using Dockerfile.serve) and push it to Artifact Registry as the first two step of the cloudbuild_serve.yaml file.
- 3** Deploy the Flask app to Cloud run
The flask app on the image gets deployed to Cloud Run, which fetch the Endpoint Id and serve online prediction.



Data Loading: The Foundation of the Model

Essential steps for effective data ingestion

The **data_loader.py** script is crucial for the app, as it efficiently ingests data from cloud bucket. It ensures that raw data is retrieved, enabling subsequent processing steps (dropping irrelevant features) to build a robust flight price prediction model.

Data Cleaning: Ensuring Data Quality

Transforming raw data into a usable format

The **data_cleaner.py** script plays a vital role in our pipeline. It fills the null values (with median for numerical column and with mode for categorical column), ensuring that only high-quality data feeds into the model. This step is crucial for achieving accurate predictions in our flight price prediction app.





Feature Engineering Techniques

Transforming raw data into usable feature

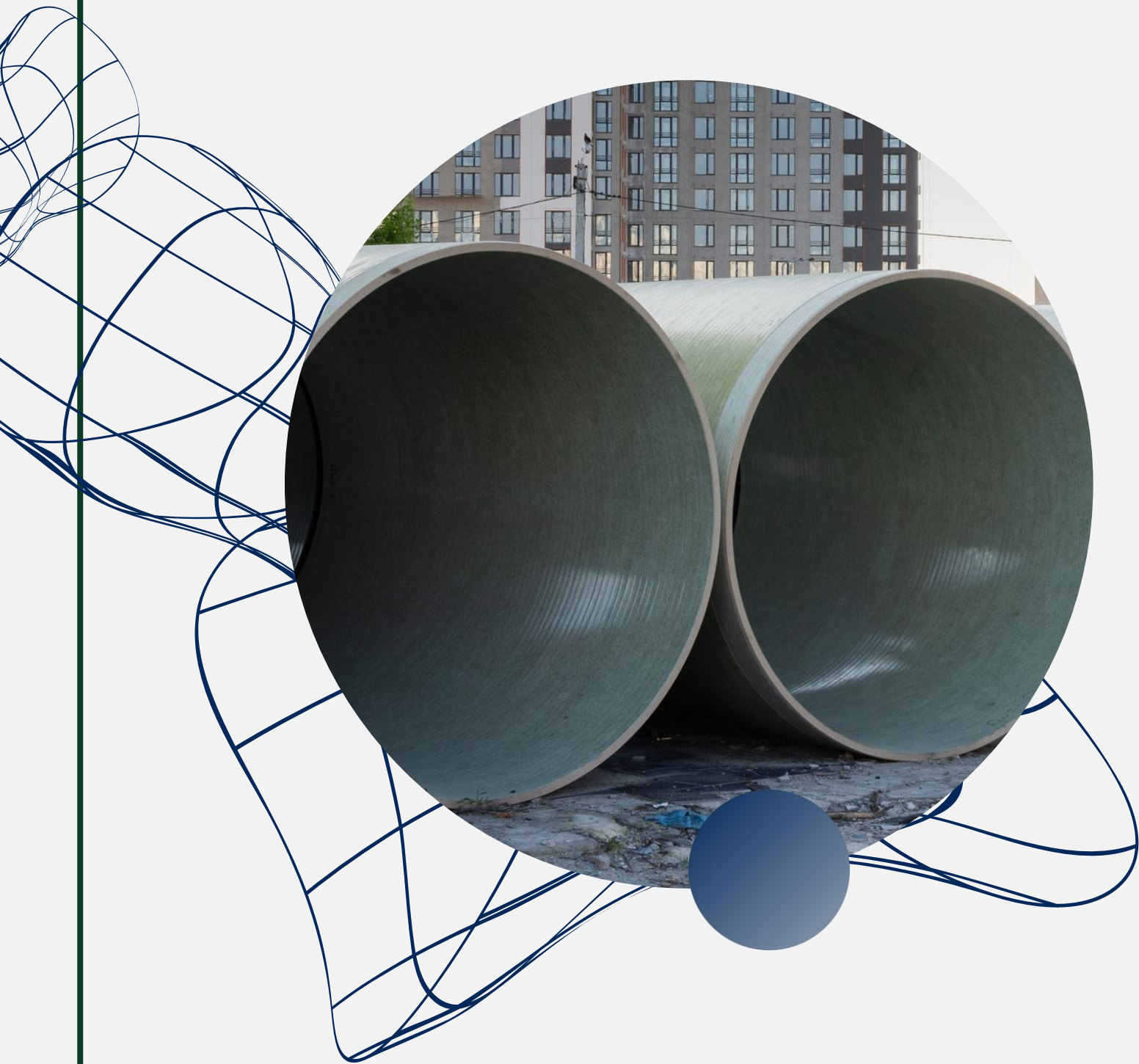
The **feature_engineering.py** script enhances model performance by transforming and selecting the most relevant features from the dataset. It perform Standard Scaling on Numerical features and One Hot Encoding on Categorical features. This process is crucial for improving predictive accuracy and ensuring that the model learns effectively from the given data.



Model Training Process Overview

Optimizing models for accurate flight price predictions

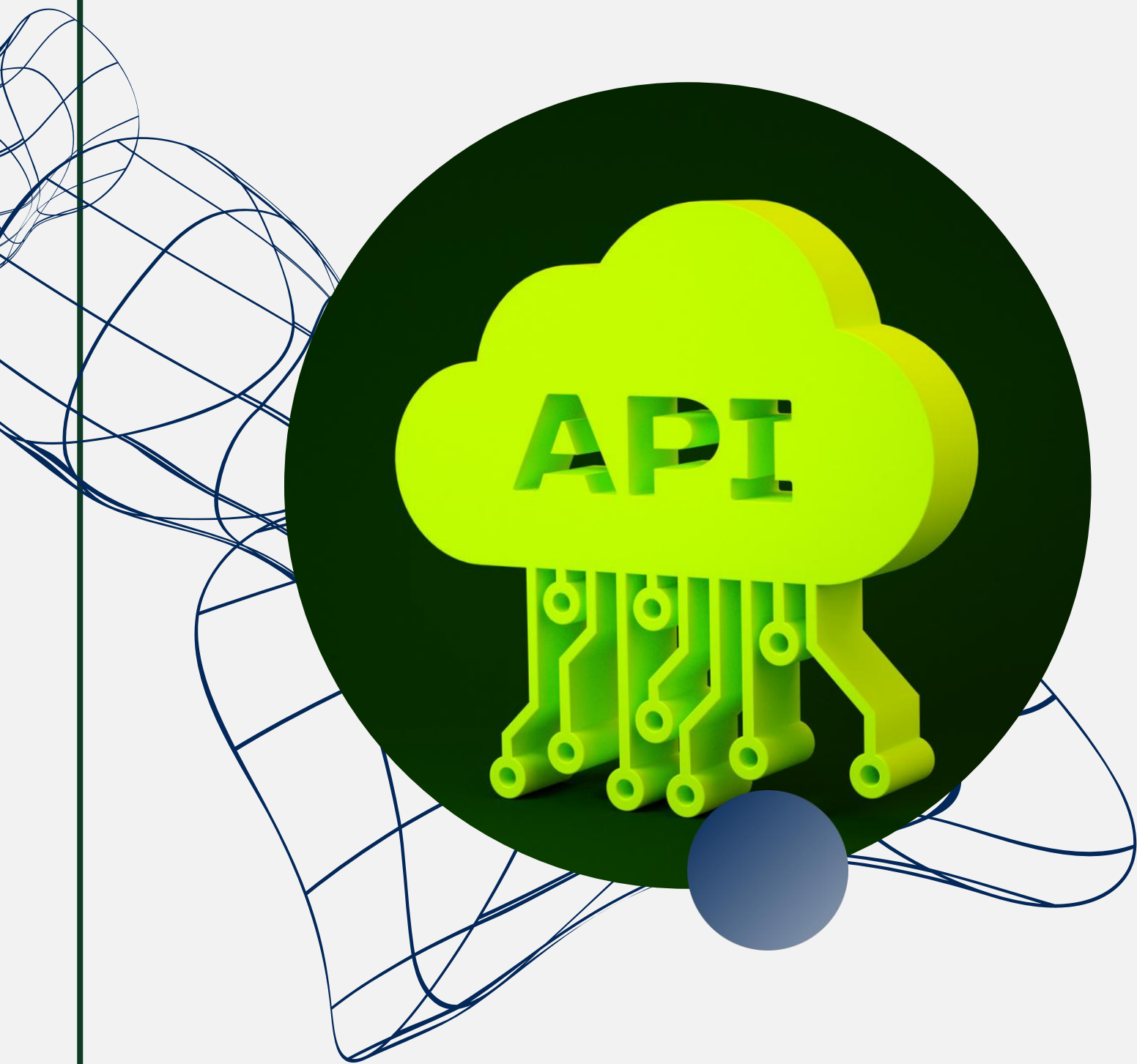
The **model_trainer.py** script is essential for training and optimizing the prediction engine. It utilizes Decision Tree Regressor algorithms with Hyperparameter tuning to enhance accuracy, enabling the application to predict flight prices effectively. This process is crucial for providing users with reliable information.



Main Orchestration of the Pipeline

Integrating classes for efficient model training

The **main.py** file serves as the central hub, and entry point training orchestrating the workflow by connecting all components of the project. It ensures that data flows seamlessly from loading to cleaning, feature engineering, and model training, creating an efficient pipeline for deployment.



Real-Time Predictions with Flask

Utilizing Flask for efficient API interactions

The **run.py** file serves as the backbone of the application, providing an API endpoint to query flight prices. This allows users to obtain real-time predictions seamlessly, enhancing the overall user experience in accessing flight price information.

Dockerizing the Application

Ensures **consistent environments** for reliable training and prediction processes.

Dockerizing the Training pipeline

As mentioned before main.py file has been used as the entry point for the training pipeline.

```
Dockerfile.train > ...
1 FROM python:3.10-slim
2
3 WORKDIR /app
4 COPY app/ .
5 COPY requirements.txt ./requirements.txt
6
7 ENV PORT=8080
8
9 RUN pip install --no-cache-dir -r requirements.txt
10
11 ENTRYPOINT ["python3","main.py"]
```

Dockerizing the Serving pipeline

The Serving pipeline is designed in run.py file, containing the Flask app which serves the prediction by using the Endpoint Id.

```
Dockerfile.serve > ...
1 FROM python:3.10-slim
2
3 WORKDIR /app
4
5 COPY requirements.txt ./requirements.txt
6 RUN pip install --no-cache-dir -r requirements.txt
7
8 COPY run.py ./run.py
9 COPY templates/ ./templates
10 COPY static/ ./static
11 COPY app/ ./app
12
13 ENV PORT=8080
14 CMD exec gunicorn --bind :$PORT --workers 1 --threads 8 --timeout 0 run:app
```



Key Features of the Deployment

Overview of the app's capabilities and benefits

Deployment of the Training Pipeline

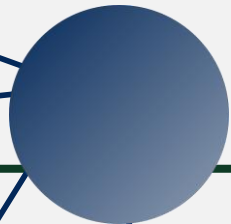
The deployment process of the Training pipeline is configured on the `cloudbuild_hpt_train.yaml` file. Which use `Dockerfile.train` for containerization.

Deployment of the Serving Pipeline

The deployment process of the Serving pipeline is configured on the `cloudbuild_serve.yaml` file. Which use `Dockerfile.serve` for containerization.

Seamless Integration

Incorporates CI/CD for smooth integration and deployment using cloud build trigger on new push to GitHub Repository.



Batch Inference

For Batch inference Python SDK for Google Cloud has been used. It is designed such a way that it will save the prediction output in the storage bucket. Please refer the 'python_sdk_batch_inference.ipynb' notebook for more details.

Workflow:

1

Upload Batch Data to storage bucket
Upload batch data in CSV format to the storage bucket.

2

Load Preprocessor and process the uploaded batch data
Load the already saved preprocessor from the storage bucket and feed the uploaded data location to the it to preprocess the data.

3

Load the Model from Model Registry
Load the Model from the Model Registry and use that for batch prediction.

```
def load_preprocessor_from_gcs(bucket_name, blob_path):
    #Download preprocessor from GCS and load it
    client = storage.Client()
    bucket = client.bucket(bucket_name)
    blob = bucket.blob(blob_path)

    with tempfile.NamedTemporaryFile(delete=False) as temp_file:
        blob.download_to_filename(temp_file.name)
        preprocessor = joblib.load(temp_file.name)

    return preprocessor

def process_and_upload_file_to_gcs(gcs_input_uri, preprocessor, bucket_name, source_file, destination_blob):

    os.makedirs("../processed_batch_data", exist_ok=True)

    df = pd.read_csv(gcs_input_uri)
    processed_data = preprocessor.transform(df)
    transformed_df = pd.DataFrame(processed_data)
    transformed_df.to_csv("../processed_batch_data/transformed_input.csv", index=False)

    storage_client = storage.Client()
    bucket = storage_client.bucket(bucket_name)
    blob = bucket.blob(destination_blob)
    blob.upload_from_filename(source_file)

    transformed_data_gcs_uri = f"gs://{bucket_name}/{destination_blob}"
    return transformed_data_gcs_uri
```

```
gcs_input_uri = 'gs://flight_price_data/data/airlines_flights_data_small_batch.csv'
BUCKET_URI = "gs://flight_price_data/output"

PREPROCESSOR_BLOB_PATH = f"{PROCESSOR_DIR}/{PROCESSOR_FILENAME}"
preprocessor = load_preprocessor_from_gcs(GCS_BUCKET_NAME, PREPROCESSOR_BLOB_PATH)

model = aiplatform.Model(model_name="projects/flight-price-prediction-470515/locations/us-central1/models/3970181456759619584")

transformed_data_gcs_uri = process_and_upload_file_to_gcs(
    gcs_input_uri, preprocessor, GCS_BUCKET_NAME,
    "processed_batch_data/transformed_input.csv", "data/transformed_input.csv")

batch_predict_job = model.batch_predict(
    job_display_name="Flight_Price_Prediction_Batch_Job",
    gcs_source=transformed_data_gcs_uri,
    gcs_destination_prefix=BUCKET_URI,
    instances_format="csv",
    predictions_format="jsonl",
    machine_type="n1-standard-4",
    starting_replica_count=1,
    max_replica_count=1,
    service_account="vertexai-sa@flight-price-prediction-470515.iam.gserviceaccount.com",
    generate_explanation=True,
    sync=False
)
```

Performance Monitoring

Once the Hyperparameter tuning is completed, it saves each model for each train in the storage bucket. Performance for each model also can be tracked from Google Cloud Vertex AI console and the Hyperparameters can be tuned further if needed.

Hyperparameter tuning trials

Filter

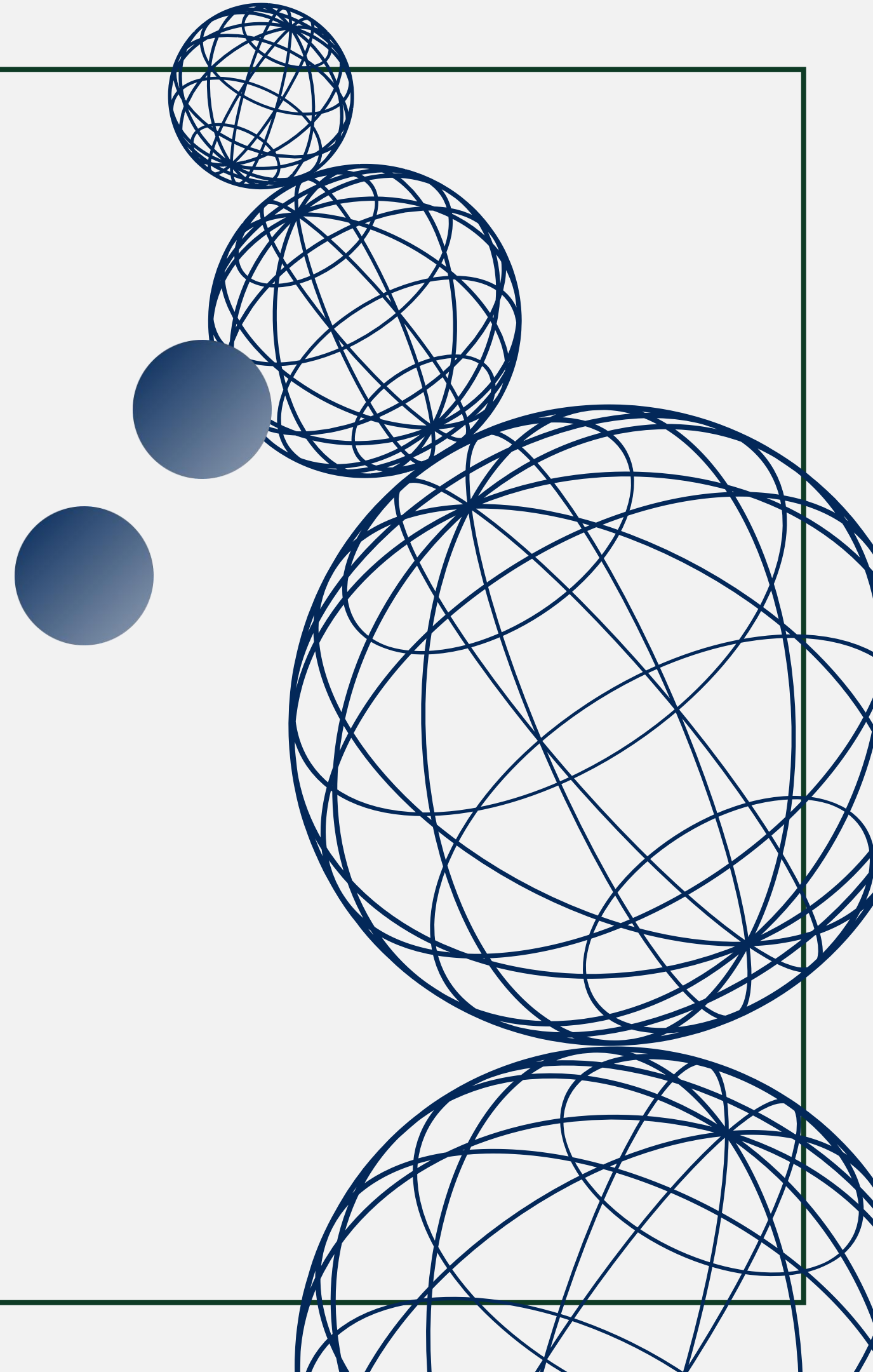
Enter property name or value

☐	Trial ID	rmse ↑	Training step	max_depth	min_samples_split	min_samples_leaf	Log
☐	5	3,010.459	1	28	2	5	View logs
☐	4	3,062.508	1	29	5	3	View logs
☐	2	3,091.959	1	25	4	3	View logs
☐	6	3,171.567	1	24	9	1	View logs
☐	1	3,231.582	1	20	6	3	View logs
☐	3	3,767.103	1	15	4	3	View logs

What are your questions?

I'd love to hear your thoughts and ideas!
Submit your questions here

Ask Your Questions





**Get in touch for more
information!**

Email

debojyotichattoraj1996@gmail.com

GitHub

[MLOps](#)

